

스프린트 미션 16: 모델 변환 및 양자화 보고서

모델 변환 및 양자 변환 보고서

연구기간: 2025.12.09 ~ 2025.12.12

연구자: 김명환 (AI 4기 3팀)

완료일: 2025.12.12

1. 미션 개요

1.1. 미션 목표

이전 미션에서 학습한 모델을 다양한 포맷으로 변환하고 양자화를 적용하여, 모델 경량화 및 추론 최적화를 실습한다.

1.2. 수행 작업

- 대상 모델: YOLOv8m (Oxford-IIIT Pet Dataset 2-class 분류)
 - 학습 환경: Google Colab L4 GPU
 - 평가 환경: Windows 11 CPU (Intel)
 - 변환 포맷:
 1. PyTorch .pt (baseline)
 2. ONNX FP32 (32-bit floating point)
 3. ONNX FP16 (16-bit floating point, half-precision)
 4. ONNX INT8 (QDQ, Quantize-Dequantize)
 5. OpenVINO INT8 (Intel CPU 최적화)
-

2. YOLOv8m 모델 구조

2.1. 모델 개요

YOLOv8m은 Ultralytics의 중형 객체 탐지 모델로, 정확도와 속도의 균형을 맞춘 모델이다.

2.2. 주요 구성 요소

- Backbone: CSPDarknet 기반 특징 추출기
 - Conv + SiLU(Swish) activation
 - C2f 모듈 (Cross-Stage Partial with 2 convolutions, faster)
- Neck: FPN + PAN 구조
- Head: Decoupled head (분리형 헤드)
 - Classification head
 - Bounding box regression head

2.3. 모델 통계 (FP32 기준)

항목	값
전체 노드 수	400 개
Conv 레이어	84 개
Activation (Mul+Sigmoid)	160 개
Concat	39 개
입력 shape	[batch, 3, 640, 640]
출력 shape	[batch, 6, 8400]
모델 크기	98.72 MB

3. 양자화 방법론

3.1. OpenVINO INT8 양자화

3.1.1. 개요

Intel OpenVINO는 Intel CPU에 최적화된 추론 프레임워크로, PTQ (Post-Training Quantization)를 통해 INT8 양자화를 지원한다.

3.1.2. 적용 방법

```
# Ultralytics 내장 기능 사용
model.export(
    format='openvino',
    int8=True,
    imgsz=640,
    data=yaml_path, # calibration data
)
```

3.1.3. 특징

➤ 장점:

- ✧ 라이브러리 버전만 맞추면 간단히 적용 가능
- ✧ Ultralytics 내장 지원으로 원클릭 양자화
- ✧ Intel CPU에서 최적 성능

➤ 단점:

- ✧ Intel 하드웨어에 종속적
- ✧ 내부 동작 커스터마이징 어려움

3.2. ONNX Runtime Static Quantization (INT8 QDQ)

3.2.1. 개요

ONNX Runtime 의 정적 양자화 (Static Quantization)는 QDQ (Quantize-Dequantize) 노드를 삽입하여 INT8 연산을 수행한다. 본 실험에서는 PTQ (Post-Training Quantization) 방식을 적용하였다.

3.2.2. 양자화 기법: PTQ (Post-Training Quantization)

- **정의:** 학습이 완료된 FP32 모델의 가중치와 활성화 값을 INT8 로 변환
- **Calibration:** 실제 데이터셋 이미지 500 장을 사용하여 활성화 값의 분포를 측정
- **장점:** 재학습 불필요, 빠른 변환
- **단점:** QAT (Quantization-Aware Training) 대비 정확도 소폭 하락 가능

3.2.3. 양자화 과정

Step 1: FP32 ONNX 모델 준비

```
# PyTorch .pt → ONNX FP32 변환
model.export(format='onnx', imgsz=640)
```

Step 2: Calibration Data Reader 구현

```
class RealDataCalibrationReader(CalibrationDataReader):
    def __init__(self, image_files, max_images=500):
        # Oxford Pet 데이터셋에서 500 장 샘플링
        # YOLO 전처리 파이프라인 적용:
        # 1. Letterbox resize (640x640, 비율 유지)
        # 2. BGR → RGB 변환
        # 3. 정규화 [0, 255] → [0, 1]
        # 4. HWC → CHW, numpy → float32
```

Step 3: 양자화 대상 레이어 선택

- **양자화 적용:**
 - ✧ Conv: 합성곱 레이어 (84 개)
 - ✧ MatMul, Gemm: 행렬 곱셈
 - ✧ Add: Residual connection
 - ✧ Mul: 요소별 곱셈
- **양자화 제외:**
 - ✧ Concat: 출력 레이어 보호 (정밀도 유지)
 - ✧ Reshape, Transpose: 구조 변경 연산
 - ✧ Sigmoid: 비선형 활성화 (양자화 효과 제한적)

Step 4: Static Quantization 실행

```

quantize_static(
    model_input=onnx_fp32_path,
    model_output=onnx_int8_path,
    calibration_data_reader=calibration_reader,
    quant_format=QuantFormat.QDQ, # CPU 호환 형식
    activation_type=QuantType.QInt8,
    weight_type=QuantType.QInt8,
    nodes_to_exclude=nodes_to_exclude,
    per_channel=True, # 채널별 양자화 (정밀도 향상)
    reduce_range=False,
    extra_options={
        'ActivationSymmetric': False, # 활성화: asymmetric
        'WeightSymmetric': True,      # 가중치: symmetric
    }
)

```

3.2.4. 주요 이슈 및 해결 과정

이슈 1: 출력 레이어 양자화 문제

- 증상: 초기 양자화 시 출력 직전 Concat 노드까지 양자화되어 추론 오류 발생
- 원인: ONNX Runtime의 기본 동작이 모든 레이어를 양자화하려 시도
- 해결: nodes_to_exclude 파라미터로 출력 레이어 체인 보호

```

# 출력 노드 역추적하여 제외 목록 생성
output_node_name = "output0"
nodes_to_exclude = ["/model.22/Concat_25", ...]

```

이슈 2: 전처리 불일치 경고

WARNING - Please consider pre-processing before quantization

- 원인: Calibration 시 전처리 파이프라인이 실제 추론과 다를 경우 정확도 저하
- 해결:
 1. YOLO 공식 전처리와 동일한 파이프라인 구현
 2. Letterbox resize + 정규화 순서 엄격히 준수
 3. BGR→RGB 변환 주의

이슈 3: QDQ 노드 폭발적 증가

- 증상: FP32 400 개 노드 → INT8 1,228 개 노드 (3 배 증가)
- 원인: 각 양자화 대상 텐서마다 QuantizeLinear + DequantizeLinear 쌍 삽입
 - ✧ QuantizeLinear: 330 개
 - ✧ DequantizeLinear: 498 개
- 영향:
 - ✧ 모델 크기는 감소 (98.72MB → 25.30MB)
 - ✧ 그래프 복잡도 증가로 추론 시간 증가 (FP32 대비 느림)

3.2.5. 양자화 가능/불가능 레이어

레이어 타입	양자화 가능 여부	이유
Conv	가능	행렬 연산, 양자화 효과 우수
MatMul/Gemm	가능	대부분의 연산량 차지
Add	가능	Residual connection, 정밀도 유지 가능
Mul	가능	SiLU activation 의 일부
Sigmoid	제한적	비선형 활성화, 양자화 시 정밀도 손실
Concat	불가	출력 레이어, 정밀도 필수
Reshape/Transpose	불가	구조 변경 연산, 양자화 의미 없음
MaxPool	제한적	비선형 연산, 양자화 효과 미미

3.2.6. 개선 방향

1. 모델 구조 분석을 통한 최적화

- 방법: 양자화 전후 ONNX 그래프 시각화 및 레이어별 영향도 분석
- 도구: onnx.helper.printable_graph(), Netron
- 목적: 병목 구간 식별, 불필요한 QDQ 노드 제거

2. Calibration 데이터셋 품질 개선

- 현재: 랜덤 샘플링 500 장
- 개선안:
 - ✧ 클래스 균형 샘플링
 - ✧ 다양한 조명/배경 조건 포함
 - ✧ 객체 크기 분포 고려

3. Per-channel vs Per-tensor Quantization

- Per-channel (현재 적용):
 - ✧ 각 출력 채널마다 별도의 scale/zero-point
 - ✧ 정밀도 높음, 연산량 증가
- Per-tensor:
 - ✧ 전체 텐서에 하나의 scale/zero-point
 - ✧ 빠름, 정밀도 다소 낮음

4. QAT (Quantization-Aware Training) 고려

- 장점: 양자화 오차를 학습 중 보정, 정확도 향상
 - 단점: 재학습 필요, 시간 소요 증가
-

4. 실험 결과

4.1. 모델 크기 비교

이름	크기	유형	수정한 날짜
mission_16_yolo8m_fp16.onnx	101,086KB	ONNX 파일	2025-12-08 월 13:40
mission_16_yolo8m_fp32.onnx	101,086KB	ONNX 파일	2025-12-08 월 13:39
mission_16_yolo8m_best.pt	50,790KB	PT 파일	2025-12-08 월 15:38
mission_16_yolo8m_int8_qdq.onnx	25,911KB	ONNX 파일	2025-12-08 월 15:26
mission_16_yolo8m_best_int8_openvino_best.bin	25,449KB	BIN 파일	2025-12-08 월 15:39

모델 포맷	파일명	파일 크기 (MB)	압축률	비고
PyTorch .pt (baseline)	mission_16_yolo8m_best.pt	49.60	-	state_dict 형식
ONNX FP32	mission_16_yolo8m_fp32.onnx	98.72	-	그래프 포함
ONNX FP16	mission_16_yolo8m_fp16.onnx	98.72	0%	FP16 변환 실패 (FP32 변환)
ONNX INT8 (QDQ)	mission_16_yolo8m_int8_qdq.onnx	25.30	74.4%	QDQ 노드 포함
OpenVINO INT8	mission_16_yolo8m_best_int8_openvino_best.bin + .xml	25.46	74.2%	Intel 최적화

압축률 계산 기준: FP32 대비 (98.72 MB 기준)

4.2. 성능 평가 결과

평가 데이터셋: Oxford-IIIT Pet Dataset (test split, 368 장)

모델	mAP50	mAP50-95	Precision	Recall	추론 시간 (ms)
YOLOv8m_baseline (.pt)	0.9942	0.9178	0.9942	0.9783	1101.03
YOLOv8m_ONNX_FP32	0.9942	0.9178	0.9942	0.9783	926.84
YOLOv8m_ONNX_FP16	0.9942	0.9178	0.9942	0.9783	831.75
YOLOv8m_int8_openvino	0.9942	0.9105	0.9917	0.9776	266.57
YOLOv8m_ONNX_int8_qdq	0.9938	0.8667	0.9881	0.9783	1399.70

4.3. 주요 분석

4.3.1. 정확도 분석

- FP32/FP16: 완전히 동일한 성능 (차이 없음)
 - ✧ FP16 변환 실패(FP32 변환): 파일 크기 확인 결과 FP32 와 동일 (98.72 MB)
 - ✧ Ultralytics 의 half=True 옵션이 CPU 환경에서 제대로 작동하지 않음
 - ✧ FP16 은 본래 GPU 에서 속도 향상되나, CPU 에서는 효과 제한적
- OpenVINO INT8:
 - ✧ mAP50: 거의 동일 (0.9942 vs 0.9942)
 - ✧ mAP50-95: 소폭 하락 (0.9178 → 0.9105, -0.8%)
 - ✧ 분석: 높은 IoU 임계값에서 약간의 정밀도 손실, 실용적으로 무시 가능
- ONNX INT8 (QDQ):
 - ✧ mAP50-95: 큰 폭 하락 (0.9178 → 0.8667, -5.6%)
 - ✧ 원인:
 - ① QDQ 노드의 양자화/역양자화 오차 누적
 - ② CPU 최적화 부족 (QDQ 는 GPU 에서 더 효과적)
 - ③ Calibration 데이터셋 대표성 부족 가능성

4.3.2. 추론 시간 분석

- OpenVINO INT8: 최고 성능 (266.57ms, 4.1 배 속도 향상)
 - ✧ Intel CPU 에 최적화된 INT8 연산 활용
 - ✧ FP32 대비 75.8% 시간 단축
- ONNX FP16: FP32 대비 10.3% 개선 (926.84 → 831.75ms)
 - ✧ 실제로는 FP16 변환이 실패(FP32 변환)했으나, ONNX Runtime 이 내부 최적화를 적용한 것으로 추정
 - ✧ CPU 에서 제한적 효과
- ONNX INT8 (QDQ): 오히려 느림 (1399.70ms, 1.5 배 증가)
 - ✧ 원인:
 - ① QDQ 노드 오버헤드 (QuantizeLinear ↔ DequantizeLinear 반복 변환)
 - ② CPU 에서 INT8 연산 최적화 부족
 - ③ 노드 수 3 배 증가 (400 → 1,228 개)

4.3.3. 모델 크기 vs 성능 트레이드오프

- OpenVINO INT8 vs ONNX INT8 QDQ:
 - ✧ 파일 크기: 거의 동일 (25.46 MB vs 25.30 MB, 0.16 MB 차이)
 - ✧ 추론 속도: OpenVINO 가 5.2 배 빠름 (266ms vs 1399ms)
 - ✧ 정확도: OpenVINO 가 5.5% 높음 (mAP50-95: 0.9105 vs 0.8667)
 - ✧ 결론: 동일한 INT8 양자화도 프레임워크에 따라 성능 차이 극명
 - ✧ Intel CPU 최적화된 OpenVINO 가 압도적 우위

4.3.4. 실용적 결론

- Windows CPU 환경:

- ✧ 추천: OpenVINO INT8 (Intel CPU 에서 최적, 속도+정확도)
 - ✧ 비추천: ONNX INT8 QDQ (CPU 에서 성능 저하)
 - GPU 환경:
 - ✧ ONNX FP16 또는 TensorRT INT8 고려
 - 정확도 우선:
 - ✧ ONNX FP32 (정확도 유지, 합리적 속도)
-

5. 결론

5.1. 핵심 성과

- YOLOv8m 모델을 5 가지 포맷으로 변환 (FP16 은 실패 - FP32 변환)
- OpenVINO INT8: 74.2% 크기 감소, 4.1 배 속도 향상, 정확도 유지
- ONNX INT8 QDQ: 74.4% 크기 감소하나 CPU 에서 추론 속도 저하 확인
- 동일 INT8 양자화도 프레임워크별 최적화 차이로 성능 5 배 이상 차이

5.2. 교훈

- 하드웨어 맞춤형 최적화 중요성: Intel CPU 에는 OpenVINO, NVIDIA GPU 에는 TensorRT
- 양자화는 은탄환이 아님: QDQ 방식은 CPU 에서 오히려 역효과
- Calibration 품질의 중요성: 전처리 파이프라인 일치 필수
- 모델 구조 분석 필수: 양자화 전후 그래프 비교를 통한 디버깅

5.3. 향후 과제

- QAT (Quantization-Aware Training) 적용하여 INT8 정확도 개선
 - TensorRT 를 활용한 GPU 최적화 비교
 - Mixed Precision (일부 레이어만 FP16/INT8) 전략 탐색
-

6. 참고 자료

6.1. 제출 파일

- 16_4 팀_김명환_학습.ipynb: YOLOv8m 모델 학습 (Colab L4)
- 16_4 팀_김명환_기본_양자화.ipynb: ONNX 및 OpenVINO 양자화 실험
- 16_4 팀_김명환_기본_평가.ipynb: 5 개 모델 통합 평가 파이프라인
- model/: 변환된 모델 파일 및 구조 분석 결과

6.2. 관련 이슈

- Ultralytics GitHub Issue #21391: ONNX INT8 지원 제한
- ONNX Runtime Documentation: Static Quantization with QDQ format
- OpenVINO Toolkit: Post-Training Optimization Tool (POT)

7. 심화

7.1. javascript 로 모델 실행 및 실행 화면

← → ↺ ⓘ 127.0.0.1:8000

🍱 | 📁 ZEP 📁 코드잇 📁 Google드라이브 📁 학습 📁 데이터 📁 Copilot 📁 결제 📁 git 🗨️ Discord 📅 Google Calendar 🌐

YOLOv8m ONNX 웹 추론 테스트

Oxford-IIIT Pet Dataset 2-class 분류 (cat, dog)

📄 **사용 방법**

- ONNX 모델 선택:** mission_16_yolo8m_fp32.onnx 또는 mission_16_yolo8m_int8_qdq.onnx
- 이미지 선택:** 고양이 또는 개 이미지 (JPG/PNG)
- 추론 실행 버튼 클릭**

⚙️ **설정:** Confidence: 0.25 | IoU: 0.45 | Input: 640x640 | Classes: 2 (cat, dog)

ONNX 모델 선택 (.onnx):

파일 선택

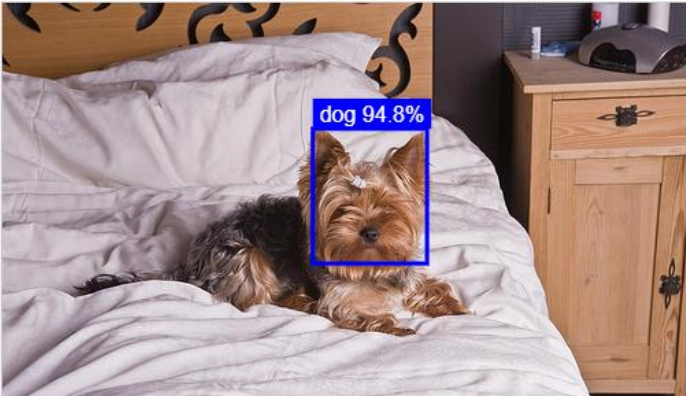
 mission_16_y...int8_qdq.onnx 이미지 파일 선택 (JPG/PNG):

파일 선택

 sample.jpg

추론 실행

상태: ✅ 추론 완료! 1개 객체 감지됨.



➤ 데모 시연

✧ <https://youtu.be/rCqQKacikjw>